1. Create a new GitHub repository.

- Clone the repository to your local machine using SSH (generate an SSH key if needed, add the public key to your GitHub account).
- Create a new branch named after your username (e.g., `Tutedude`).
- Add your Flask project files to this branch.
- Commit the changes and merge the branch into the `main` branch.

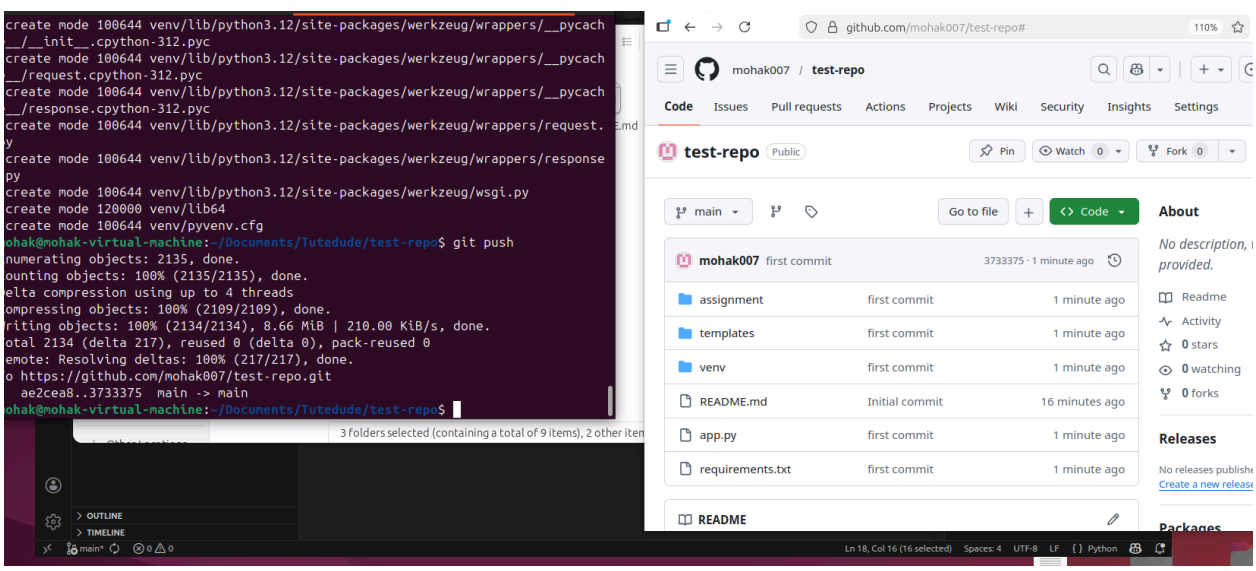
I have cloned the repo using the ssh key. For that i needed to created the pub and private key. Next I added the public key in the github ssh keys section and then created a repo in github gui. Now i have cloned the repo ‘test-repo’. Copied my flask codebase in this repo.
Then commands used

git init -> create .git file where git will keep a track of files going in and out of my repo

git add . -> it add the files in the .git file. Since i have mentioned “.” it will add all the files details into the .git file.

git commit -m “first commit” -> this is the message that will be shown on the github branch like what things were done on this commit.

git push -> will push the files after reading the information of files from the .git file.



2. Create a new branch named `<your_name>_new` (e.g., `Tutedude_new`).

- Update the content of the JSON file used for the `/api` route in this branch.
- Merge the `<your_name>_new` branch into the `main` branch.
- If there are conflicts during the merge, resolve them by accepting the changes from the `<your_name>_new` branch.
- Add the resolved changes to the staging area, commit them, and push the updates to the remote repository.

I have created a new branch called mohak_new using command git branch branchname.
Verified if the branch is created or not, from git branch command

Moved to mohak_new using git checkout branchname command

Updated the data.json and added a new key with value

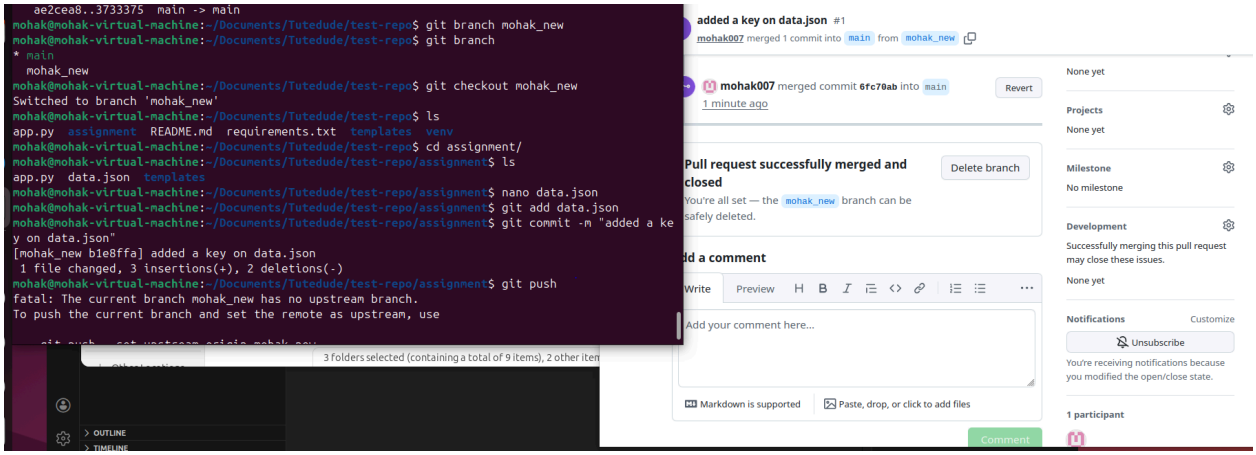
Added the new changes of that file in .git using git add filename

Committed my changes using git commit -m “added a key on data.json”

Since i need to set the upstream of the push so used git push –set-upstream origin mohak_new

so there was not merge conflicts for me

Shared the snippet of the PR as well which is merged by me only in UI

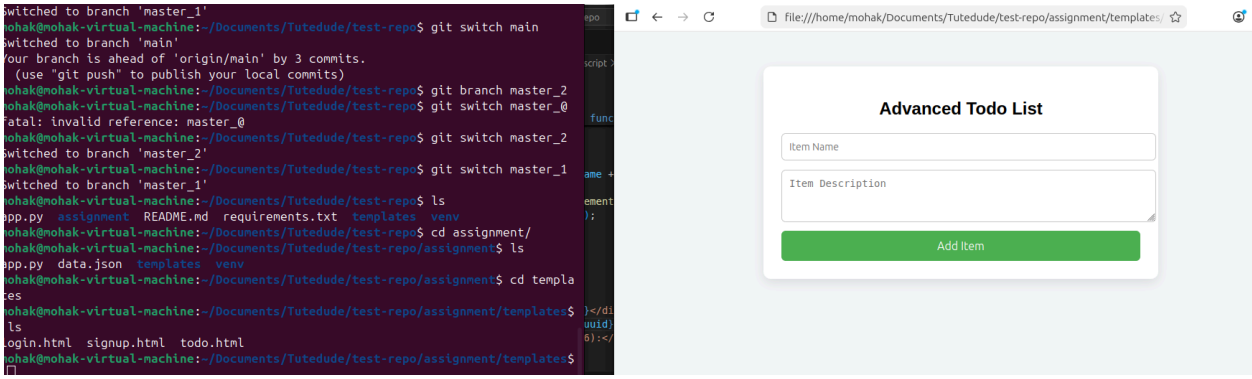


```
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin mohak_new

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.

mohak@mohak-virtual-machine:~/Documents/Tutedude/test-repo/assignment$ git push --set-upstream o
rigin mohak_new
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 454 bytes | 454.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'mohak_new' on GitHub by visiting:
remote:      https://github.com/mohak007/test-repo/pull/new/mohak_new
remote:
```

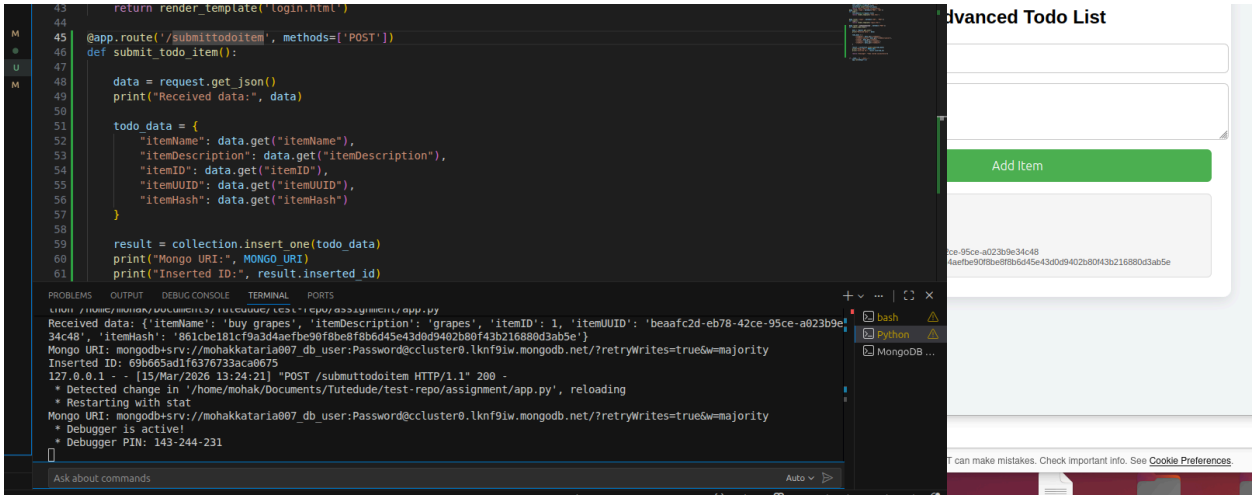
3. **Branch Creation**:

- Create two branches: `master_1` and `master_2` from the `main` branch.
- **Feature Development in** `master_1`:
- In the `master_1` branch, create a **To-Do Page** in the frontend.
    - The page should contain a form with the following fields:
        - **Item Name**
        - **Item Description**
- **Backend API in** `master_2`:
- In the `master_2` branch, create a backend route named `/submittodoitem`.
- This route will:
    - Accept `itemName` and `itemDescription` via a POST request.
    - Store these details in a MongoDB database.
- **Merging Changes**:
- Merge the changes from both `master_1` and `master_2` into the `main` branch.

I have created both the branches master_1 and master_2 using the git branch. And used git switch to go to those branches.
I have created a todo.html in the master_1 branch.



In branch master_2 I have added the submit to do item backed api to mongodb and was able to receive the data.



Merged changes from master_1 to main

added todo html #8

Code

Merged mohak007 merged 1 commit into main from master_1 now

Conversation 0 | Commits 1 | Checks 0 | Files changed 2 | +62 -23

mohak007 commented now — Owner

No description provided.

added todo html 26ca0c0

mohak007 merged commit **7088295** into main now — Revert

Pull request successfully merged and closed
You're all set — the master_1 branch can be safely deleted.

Delete branch

Add a comment

Write | Preview

Add your comment here...

Reviewers
No reviews
Still in progress? Convert to draft

Assignees
No one—assign yourself

Labels
None yet

Projects
None yet

Milestone
No milestone

Development
Successfully merging this pull request may close these issues.
None yet

My Queries
Data Modeling

CLUSTERS (1)
Search clusters
Cluster0
admin
local
sample_mflix
test
flask-tutorial
flask-tutorial-assignment

Cluster0 > test > flask-tutorial-assignment

View monitoring | Visualize Your D

Documents 2 | Aggregations | Schema | Indexes 1 | Validation

Type a query: { field: 'value' } or Generate query

Explain | Reset | Find | Options

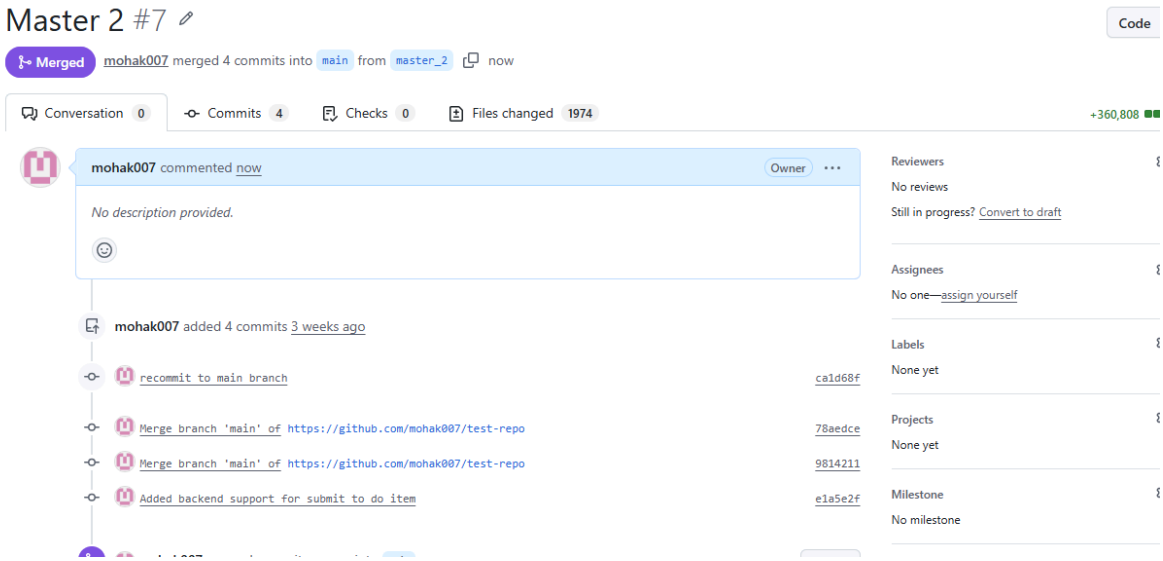
ADD DATA | UPDATE | DELETE | EXPORT CODE

25 | 1 - 4 of 4

```
_id: ObjectId('699980b0f2a022f2bc61ae9a')

_id: ObjectId('699980f8838c4fde18d7d163')
fullname : "mohak"
email : "mohak@mohak.com"
password : "mohak"
confirm_password : "mohak"

_id: ObjectId('69b6656e80fd1dacccc6317ba')
itemName : "buy grapes buy grapes from store"
itemDescription : "buy grapes"
itemID : 5
itemUUID : "7ee7482e-b1ff-4e8e-ace5-cb0df1c2c888"
itemHash : "bcb8f816edbb3257eb4bfdb7a4bdd10d524b729028cc90bed38f8ba06491c97a"

_id: ObjectId('69b665ad1f6376733aca0675')
itemName : "buy grapes"
itemDescription : "grapes"
itemID : 1
itemUUID : "beaafc2d-eb78-42ce-95ce-a023b9e34c48"
itemHash : "861cbe181cf9a3d4aefbe90f8be8f8b6d45e43d0d9402b80f43b216880d3ab5e"
```

System Status: All Good

Merged the changes of master_2 in the main



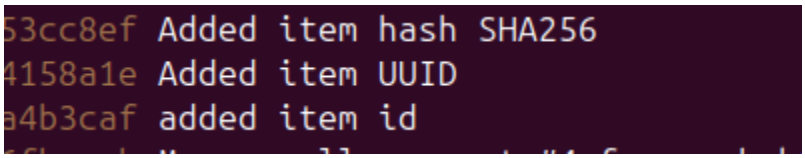
4. **Enhancing the To-Do Form in** `master_1`:

- In the `master_1` branch, add the following fields to the To-Do form:
  - **Item ID**
  - **Item UUID**
  - **Item Hash**
- **Committing in Sequence**:
- Add and commit each field separately in the following order:
  - **First commit**: Add **Item ID** field.
  - **Second commit**: Add **Item UUID** field.
  - **Third commit**: Add **Item Hash** field.
- **Merging to** `main`:
- Merge the `master_1` branch into the `main` branch.
- **Git Reset and Commit Deletion**:
- In the `main` branch, use **Git Reset** to roll back to the commit where only the **Item ID** field was added.
- Use `git reset --soft` to ensure changes remain staged.
- Re-commit this state to the `main` branch.
- Merge this updated state to the `main` branch.
- **Rebasing Changes**:
- Rebase the updated changes in the `main` branch to the `master_1` branch.
  Clarification:
  - During rebasing, **preserve individual commits** to maintain the commit history for each change (i.e., do not squash commits).

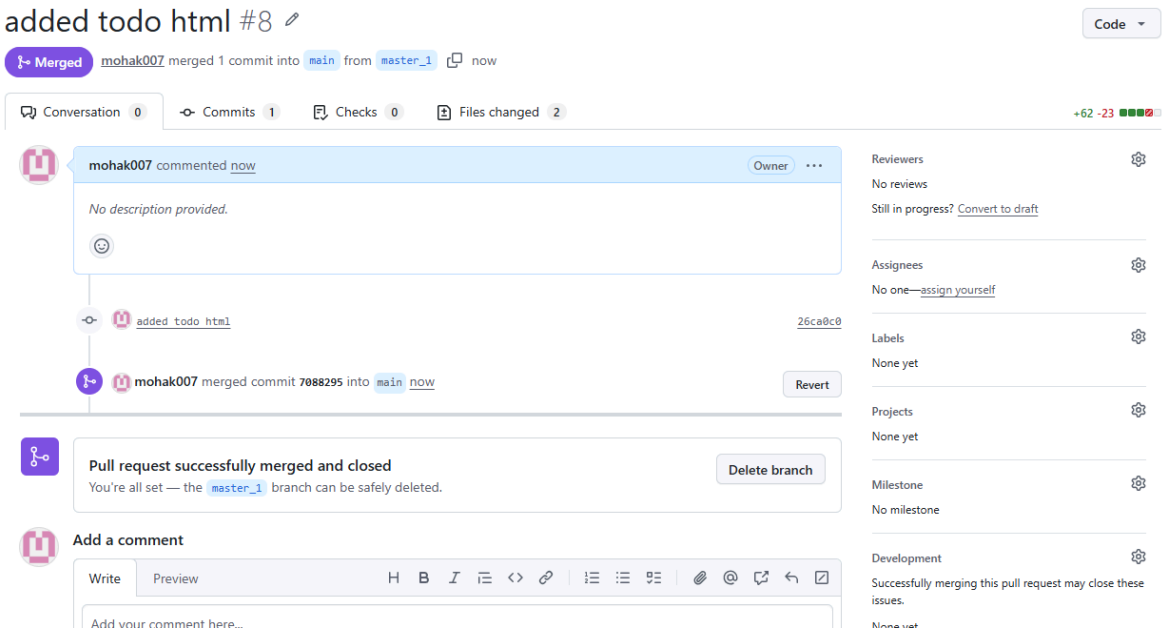
- Use `git rebase main master_1` to integrate changes from the `main` branch back into the `master_1` branch.

Added the todo_form from master_1 and merged it in the main branch.

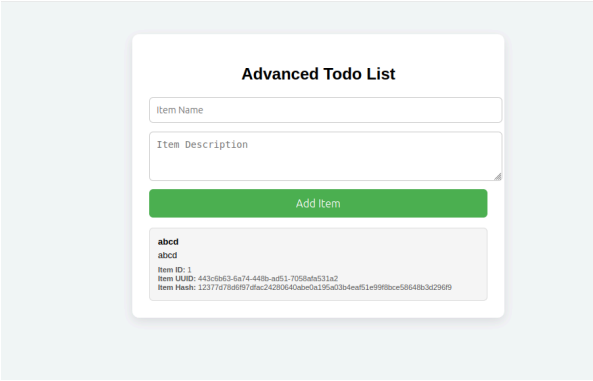
Git commit



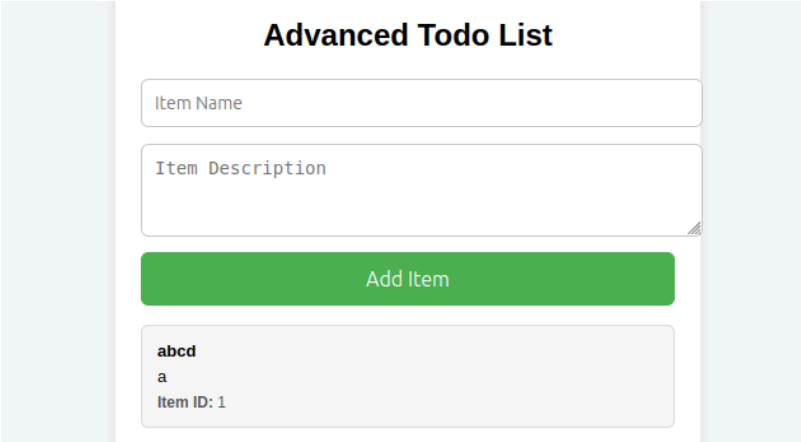
Pr details



This is the todo.html UI



After git reset to rollback to commit where only itemID was there



- Use `git reset --soft` to ensure changes remain staged.

```
mohak@mohak-virtual-machine:~/Documents/Tutedude/test-repo/assignment/templates$ git reset --soft HEAD~
2
```

- Re-commit this state to the `main` branch.

```
mohak@mohak-virtual-machine:~/Documents/Tutedude/test-repo/assignment/templates$ git commit -m "Added i
tem id filed"
[main 111c4f1] Added item id filed
 3 files changed, 141 insertions(+), 28 deletions(-)
 create mode 100644 assignment/templates/login.html
mohak@mohak-virtual-machine:~/Documents/Tutedude/test-repo/assignment/templates$ git log --online
fatal: unrecognized argument: --online
```

- **Rebasing Changes**:
- Rebase the updated changes in the `main` branch to the `master_1` branch.

```
mohak@mohak-virtual-machine:~/Documents/Tutedude/test-repo/assignment/templates$ git rebase main
fatal: It seems that there is already a rebase-merge directory, and
I wonder if you are in the middle of another rebase.  If that is the
case, please try
        git rebase (--continue | --abort | --skip)
If that is not the case, please
        rm -fr ".git/rebase-merge"
and run me again.  I am stopping in case you still have something
valuable there
```

Clarification:

- During rebasing, **preserve individual commits** to maintain the commit history for each change (i.e., do not squash commits).
- Use `git rebase main master_1` to integrate changes from the `main` branch back into the `master_1` branch.

```
mohak@mohak-virtual-machine:~/Documents/Tutedude/test-repo/assignment/templates$ git push origin main -
-force
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 4 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 2.08 KiB | 532.00 KiB/s, done.
Total 7 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/mohak007/test-repo.git
 + 34b4215...111c4f1 main -> main (forced update)
mohak@mohak-virtual-machine:~/Documents/Tutedude/test-repo/assignment/templates$ git push origin master
_1
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 4 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 2.86 KiB | 976.00 KiB/s, done.
Total 7 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
To https://github.com/mohak007/test-repo.git
mohak@mohak-virtual-machine:~/Documents/Tutedude/test-repo/assignment/templates$
```